



数値計算法 第7回

連立一次方程式 (3)

濱本 雄治¹

情報工学部 情報通信工学科

2026 年 6 月 1 日

¹hamamoto@c.oka-pu.ac.jp

復習: LU 分解による連立一次方程式の解法

LU 分解

係数行列 A を下三角行列 L と上三角行列 U の積に分解

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ a_{n1} & \cdots & \cdots & a_{nn} \end{pmatrix} = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ \ell_{21} & 1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ \ell_{n1} & \cdots & \ell_{n,n-1} & 1 \end{pmatrix} \quad (\text{下三角 } L)$$
$$\times \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \ddots & \vdots \\ \vdots & \ddots & \ddots & u_{n-1,n} \\ 0 & \cdots & 0 & u_{nn} \end{pmatrix} \quad (\text{上三角 } U)$$

復習: LU 分解による連立一次方程式の解法

L と U を係数行列とする 2 つの連立一次方程式を解く

1. LU 分解 $A = LU$ を $Ax = b$ に代入 $LUx = b$
2. $Ux = y$ において 2 つの連立一次方程式に分解

$$Ly = b \quad \dots \textcircled{1} \quad Ux = y \quad \dots \textcircled{2}$$

3. $y_1, \dots, y_{j-1}$ (昇順) が既知のとき、①の j 行目から y_j が求まる

$$\underset{\text{未知}}{y_j} = b_j - \underset{\text{既知}}{\ell_{j1}} y_1 - \dots - \underset{\text{既知}}{\ell_{j,j-1}} y_{j-1} \quad (\text{前進代入})$$

4. $x_n, \dots, x_{k+1}$ (降順) が既知のとき、②の k 行目から x_k が求まる

$$\underset{\text{未知}}{x_k} = \frac{1}{u_{kk}} (y_k - \underset{\text{既知}}{u_{kn}} x_n - \dots - \underset{\text{既知}}{u_{k,k-1}} x_{k-1}) \quad (\text{後退代入})$$

第7回の目標



- ▶ 係数行列が帯行列のときの消去法の問題を理解する
 - ▷ メモリ効率と計算量
 - ▷ 誤差と安定性
- ▶ 反復法による連立一次方程式の解法を理解する
 - ▷ Jacobi 法
 - ▷ Gauss-Seidel 法
 - ▷ SOR 法

消去法が効率的でない場合: 帯行列



▶ 特徴

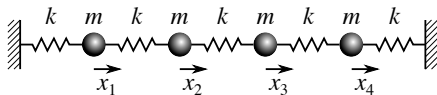
- ▷ 要素の大部分がゼロ (疎行列)
- ▷ 非ゼロ成分が対角線付近に存在

$$A = \begin{pmatrix} a_{11} & a_{12} & 0 & \cdots & 0 \\ a_{21} & a_{22} & a_{23} & \ddots & \vdots \\ 0 & a_{32} & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & a_{n-1,n-1} & a_{n-1,n} \\ 0 & \cdots & 0 & a_{n,n-1} & a_{nn} \end{pmatrix} \quad (\text{三重対角行列})$$

帯行列の例



▶ バネで繋がれた質点系



▷ 運動方程式

$$\begin{cases} m\ddot{x}_1 = -2kx_1 + kx_2 \\ m\ddot{x}_2 = kx_1 - 2kx_2 + kx_3 \\ m\ddot{x}_3 = kx_2 - 2kx_3 + kx_4 \\ m\ddot{x}_4 = kx_3 - 2kx_4 \end{cases}$$

▷ 4つの質点が同じ振動数で $x_i = X_i \sin \omega t$ のように振動するとき

$$\begin{pmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{pmatrix} \begin{pmatrix} X_1 \\ X_2 \\ X_3 \\ X_4 \end{pmatrix} = \frac{m\omega^2}{k} \begin{pmatrix} X_1 \\ X_2 \\ X_3 \\ X_4 \end{pmatrix}$$

物理の問題で現れる行列の多くは帯行列

係数行列が帯行列の連立一次方程式



▶ 帯行列の長所

- ▷ 帯部分の要素だけ記憶すればよいのでメモリ効率がよい
- ▷ 工夫すれば計算量 $\sim \mathcal{O}(N \times \text{帯の幅}) \ll \underbrace{\mathcal{O}(N^2)}_{\text{代入}} \ll \underbrace{\mathcal{O}(N^3)}_{\text{消去}}$

▶ 消去法で解く場合の問題

- ▷ ピボット選択は帯構造を破壊
→ 使用メモリと計算量が増加
- ▷ ピボット選択せず帯構造を維持
→ 丸め誤差の累積と計算の不安定化

係数行列が帯行列のときは、反復法で連立一次方程式を解く

復習：非線形方程式の反復法



原理

1. 非線形方程式 $f(x) = 0$ を次の形に変形

$$x = g(x)$$

2. 次の漸化式を用いて、初期値 x_0 から数列 $\{x_k\}$ を生成

$$x_{k+1} = g(x_k) \quad \xrightarrow{\text{生成}} \quad x_0, x_1, x_2, \dots$$

3. $\{x_k\}$ が数 α に収束すれば、 α は非線形方程式の解

▶ 例: Newton 法

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

連立一次方程式の反復法



原理

1. 連立一次方程式 $Ax = b$ を次の形に変形

$$x = Bx + c$$

2. 次の漸化式を用いて、初期値 x_0 からベクトル列 $\{x_k\}$ を生成

$$x_{k+1} = Bx_k + c \quad \xrightarrow{\text{生成}} \quad x_0, x_1, x_2, \dots$$

3. $\{x_k\}$ がベクトル α に収束すれば、 α が連立一次方程式の解

原理

- ▶ 係数 A を対角行列 D 、下三角行列 L 、上三角行列 U に分解

$$A = D + L + U$$

$$D = \begin{pmatrix} d_{11} & 0 & \cdots & 0 \\ 0 & \ddots & & \vdots \\ \vdots & & \ddots & 0 \\ 0 & \cdots & 0 & d_{nn} \end{pmatrix}, \quad L = \begin{pmatrix} 0 & \cdots & \cdots & 0 \\ \ell_{21} & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ \ell_{n1} & \cdots & \ell_{nn} & 0 \end{pmatrix}, \quad U = \begin{pmatrix} 0 & u_{12} & \cdots & u_{1n} \\ \vdots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & u_{n-1,n} \\ 0 & \cdots & \cdots & 0 \end{pmatrix}$$

- ▶ 漸化式の導出

$$(D + L + U)x = b, \quad x = D^{-1}[-(L + U)x + b],$$

$$\therefore x_{k+1} = \underbrace{-D^{-1}(L + U)}_B x_k + \underbrace{D^{-1}b}_c$$

Jacobi 法の例



▶ 2 変数の連立一次方程式

$$\begin{cases} 2x - y = 1 \\ -x + 2y = 1 \end{cases}, \quad \begin{cases} x = \frac{y+1}{2} \\ y = \frac{x+1}{2} \end{cases}$$

▶ 並列代入

$$(x, y) \leftarrow \left(\frac{y+1}{2}, \frac{x+1}{2} \right), \quad \therefore \begin{cases} x_{k+1} = \frac{y_k+1}{2} \\ y_{k+1} = \frac{x_k+1}{2} \end{cases}$$

(1) 古い x, y で右辺をすべて計算

(2) 左辺の変数にまとめて代入

Jacobi 法は並列計算に適している

Jacobi 法の計算例



▶ jacobi.c

```
#include <stdio.h>

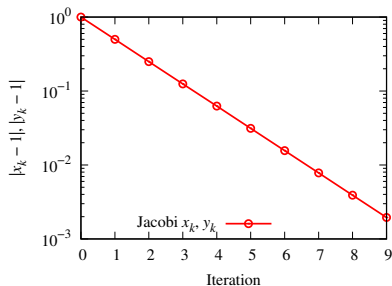
int main(){
    double x = 0;
    double y = 0;

    for(int i=0; i!=10; ++i){
        printf("%d %e %e\n", i, x, y);

        double tmp = x;
        x = (y + 1.0) / 2;
        y = (tmp + 1.0) / 2;
    }
}
```

▶ 実行結果

```
$ gcc jacobi.c; ./a.out
...
7 9.921875e-01 9.921875e-01
8 9.960938e-01 9.960938e-01
9 9.980469e-01 9.980469e-01
```



原理

- ▶ 係数行列の分解は Jacobi 法と同じ
- ▶ 漸化式の導出

$$(\textcolor{red}{D} + \textcolor{green}{L} + \textcolor{blue}{U})x = b, \quad x = (\textcolor{red}{D} + \textcolor{green}{L})^{-1}[-\textcolor{blue}{U}x + b],$$

$$\therefore x_{k+1} = -\underbrace{(\textcolor{red}{D} + \textcolor{green}{L})^{-1}\textcolor{blue}{U}}_B x_k + \underbrace{(\textcolor{red}{D} + \textcolor{green}{L})^{-1}b}_c$$

- ▶ Jacobi 法との違い

$$\triangleright \textcolor{red}{D} \rightarrow \textcolor{red}{D} + \textcolor{green}{L}$$

$$\triangleright \textcolor{green}{L} + \textcolor{blue}{U} \rightarrow \textcolor{blue}{U}$$

- ▶ Jacobi 法に類似した形で表現

$$x_{k+1} = -\textcolor{red}{D}^{-1}(\textcolor{green}{L}x_{k+1} + \textcolor{blue}{U}x_k) + \textcolor{red}{D}^{-1}b$$

Gauss-Seidel 法の例



- ▶ 2 変数の連立一次方程式 (ここまでは Jacobi 法と同じ)

$$\begin{cases} 2x - y = 1 \\ -x + 2y = 1 \end{cases}, \quad \begin{cases} x = \frac{y+1}{2} \\ y = \frac{x+1}{2} \end{cases}$$

- ▶ 逐次代入

$$x \leftarrow \frac{y+1}{2}; \quad y \leftarrow \frac{x+1}{2}, \quad \therefore \begin{cases} x_{k+1} = \frac{y_k+1}{2} \\ y_{k+1} = \frac{x_{k+1}+1}{2} = \frac{y_k+3}{4} \end{cases}$$

- (1) 古い y で $(y+1)/2$ を計算して x に代入
- (2) 新しい x で $(x+1)/2$ を計算して y に代入

y が早く更新されるため、一般に Jacobi 法より収束が早い

Gauss-Seidel 法の計算例



▶ gauss_seidel.c

```
#include <stdio.h>

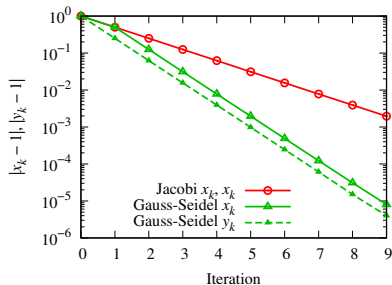
int main(){
    double x = 0;
    double y = 0;

    for(int i=0; i!=10; ++i){
        printf("%d %f %f\n", i, x, y);

        x = (y + 1.0) / 2;
        y = (x + 1.0) / 2;
    }
}
```

▶ 実行結果

```
$ gcc gauss_seidel.c; ./a.out
...
7 9.998779e-01 9.999390e-01
8 9.999695e-01 9.999847e-01
9 9.999924e-01 9.999962e-01
```



反復法の収束性



▶ 係数行列が解の収束性に影響

▷ 元の連立一次方程式

$$\begin{cases} 2x - y = 1 \\ -x + 2y = 1 \end{cases},$$

$$y_{k+1} = \frac{y_k + 3}{2}, \quad \therefore y_{k+1} - 1 = \frac{1}{4}(y_k - 1) = \cdots = \left(\frac{1}{4}\right)^{k+1} (y_0 - 1)$$

$y \rightarrow 1$ に収束

▷ 順序を入れ替えた連立一次方程式

$$\begin{cases} -x + 2y = 1 \\ 2x - y = 1 \end{cases}, \quad \therefore \begin{cases} x_{k+1} = 2y_k - 1 \\ y_{k+1} = 2x_{k+1} - 1 = 4y_k - 3 \end{cases}$$

$$\therefore y_{k+1} - 1 = 4(y_k - 1) = \cdots = 4^{k+1}(y_0 - 1)$$

$y \rightarrow \infty$ に発散

対角優位行列



▶ 係数行列 A が対角優位行列

$$|a_{ii}| > \sum_{j(\neq i)} |a_{ij}| \quad (i = 1, 2, \dots, n)$$

ならば Jacobi 法と Gauss-Seidel 法は任意の初期値に対して収束

▷ 順序の入れ替え前

$$A = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} \quad \rightarrow \quad \text{対角優位である}$$

▷ 順序の入れ替え後

$$A = \begin{pmatrix} -1 & 2 \\ 2 & -1 \end{pmatrix} \quad \rightarrow \quad \text{対角優位でない}$$

Successive Over-Relaxation (SOR) 法



原理

- ▶ Gauss-Seidel 法の漸化式

$$\mathbf{x}_{k+1}^{\text{GS}} = \mathbf{D}^{-1}[-(\mathbf{L}\mathbf{x}_{k+1} + \mathbf{U}\mathbf{x}_k) + \mathbf{b}]$$

- ▶ 修正量 $\mathbf{x}_{k+1}^{\text{GS}} - \mathbf{x}_k$ に加速パラメータ ω を掛けて収束を加速

$$\begin{aligned}\mathbf{x}_{k+1} &= \mathbf{x}_k + \omega(\mathbf{x}_{k+1}^{\text{GS}} - \mathbf{x}_k) \\ &= (1 - \omega)\mathbf{x}_k + \omega\mathbf{D}^{-1}[-(\mathbf{L}\mathbf{x}_{k+1} + \mathbf{U}\mathbf{x}_k) + \mathbf{b}],\end{aligned}$$

$$(\mathbf{D} + \omega\mathbf{L})\mathbf{x}_{k+1} = (1 - \omega)\mathbf{D}\mathbf{x}_k - \omega\mathbf{U}\mathbf{x}_k + \omega\mathbf{b},$$

$$\therefore \mathbf{x}_{k+1} = (\mathbf{D} + \omega\mathbf{L})^{-1}[-\{(\omega - 1)\mathbf{D}\mathbf{x}_k + \omega\mathbf{U}\}\mathbf{x}_k + \omega\mathbf{b}]$$

- ▷ $\omega = 1$ は普通の Gauss-Seidel 法
- ▷ $1 < \omega < 2$ 、典型的には $\omega \sim 1.9$

SOR法の例



▶ Gauss-Seidel法の漸化式

$$\begin{cases} 2x - y = 1 \\ -x + 2y = 1 \end{cases}, \quad \therefore \begin{cases} x_{k+1}^{\text{GS}} = \frac{y_k + 1}{2} \\ y_{k+1}^{\text{GS}} = \frac{x_{k+1} + 1}{2} \end{cases}$$

▶ SOR法の漸化式

$$\begin{cases} x_{k+1} = x_k + \omega(x_{k+1}^{\text{GS}} - x_k) = x_k + \omega\left(\frac{y_k + 1}{2} - x_k\right) \\ y_{k+1} = y_k + \omega(y_{k+1}^{\text{GS}} - y_k) = y_k + \omega\left(\frac{x_{k+1} + 1}{2} - y_k\right) \end{cases}$$

SOR法の計算例



▶ gauss_seidel.c

```
#include <stdio.h>

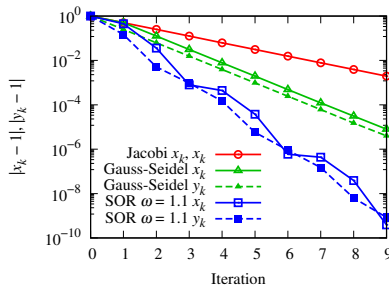
int main(){
    double x = 0;
    double y = 0;
    double w = 1.1;

    for(int i=0; i!=10; ++i){
        printf("%d %.10e %.10e\n",
               i, x, y);

        x = (1-w) * x + w * (y+1.0)/2;
        y = (1-w) * y + w * (x+1.0)/2;
    }
}
```

▶ 実行結果

```
$ gcc gauss_seidel.c; ./a.out
...
7 9.99999956491e-01 9.9999985059e-01
8 9.99999996133e-01 9.9999999367e-01
9 1.00000000004e+00 1.00000000008e+00
```



第7回のまとめ



- ▶ 係数行列が帯行列のときの消去法の問題
 - ▷ ピボット選択は帯構造を破壊 → 使用メモリと計算量を増加
 - ▷ ピボット選択せず帯構造を維持 → 誤差の蓄積、計算の不安定化
- ▶ 反復法による連立一次方程式の解法を理解する
 - ▷ Jacobi 法
 - ▷ Gauss-Seidel 法
 - ▷ SOR 法